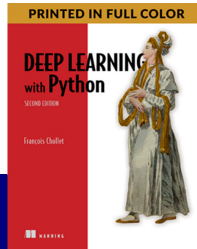


CS59300

Application of Deep Learning DL for Timeseries

Instructor: Dr. Zesheng Chen



PURDUE UNIVERSITY FORT WAYNE Outline

- Different Kinds of Timeseries Tasks
- A Temperature-forecasting Example
- Understanding Recurrent Neural Networks (RNNs)
- Advanced Use of RNNs

2

PURDUE UNIVERSITY FORT WAYNE What is a Timeseries?

- A timeseries can be any data obtained via measurements at **regular intervals**, such as
 - The daily price of a stock
 - The hourly electricity consumption of a city
 - The weekly sales of a store.
- Working with timeseries involves understanding the **dynamics** of a system
 - Its periodic cycles
 - How it trends over time
 - Its regular regime and its sudden spikes

3

PURDUE UNIVERSITY FORT WAYNE What are Different Kinds of Timeseries Tasks?

- Forecasting
 - Predict what will happen next in a series
 - **Focus** in this chapter
- Classification
 - Assign one or more **categorical** labels to a timeseries
 - E.g., classify whether the visitor is a bot or a human
- Event Detection
 - Identify the **occurrence** of a specific expected event within a continuous data stream
 - “Ok Google” or “Hey Alexa”
- Anomaly Detection
 - Detect anything **unusual** happening within a continuous datastream
 - Unusual activity on a corporate network

4



Domain-Specific Data Representation

- When working with timeseries, you'll encounter a wide range of **domain-specific** data representation techniques.
 - For example, **Fourier transform**
 - Map from time domain to frequency domain
 - A form of **feature engineering**
- In this chapter, we will instead focus on the modeling part.
 - Recurrent neural networks (RNNs)

5



Outline

- Different Kinds of Timeseries Tasks
- **A Temperature-forecasting Example**
- Understanding Recurrent Neural Networks (RNNs)
- Advanced Use of RNNs

6



A Temperature-forecasting Example

- Problem
 - **Predicting** the temperature 24 hours in the future, given a timeseries of hourly measurements of quantities such as atmospheric pressure and humidity, recorded over the recent past by a set of sensors on the roof of a building
- Dataset recorded at the weather station at the Max Planck Institute for Biogeochemistry in Jena, Germany
 - Every 10 minutes over 2009 - 2016

7



DL Models to Be Tried

- Densely connected networks
- Convolution neural networks (CNN)
- Recurrent neural networks (RNN)

- What is the baseline?

8



Outline

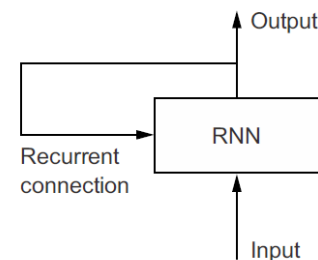
- Different Kinds of Timeseries Tasks
- A Temperature-forecasting Example
- Understanding Recurrent Neural Networks (RNNs)
- Advanced Use of RNNs

9



What is a RNN?

- A RNN processes sequences by **iterating** through the sequence elements and maintaining a **state** that contains information relative to what it has seen so far.



10



Pseudocode for RNN

```

state_t = 0
for input_t in input_sequence:
    output_t = f(input_t, state_t)
    state_t = output_t

state_t = 0
for input_t in input_sequence:
    output_t = activation(dot(W, input_t) + dot(U, state_t) + b)
    state_t = output_t
  
```

11



NumPy Implementation of a Simple RNN

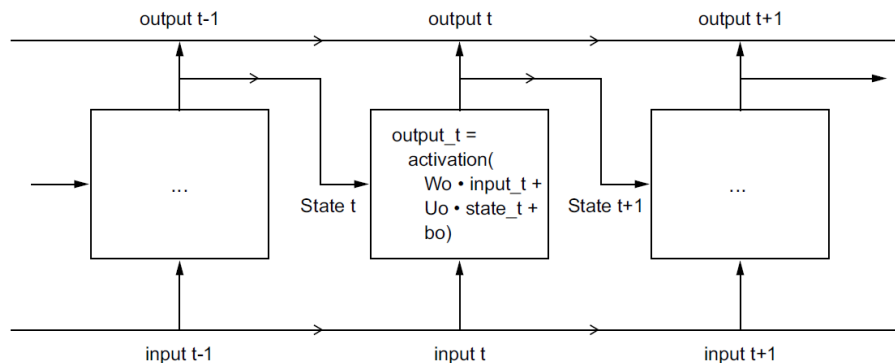
```

import numpy as np
timesteps = 100
input_features = 32
output_features = 64
inputs = np.random.random((timesteps, input_features))
state_t = np.zeros((output_features,))
W = np.random.random((output_features, input_features))
U = np.random.random((output_features, output_features))
b = np.random.random((output_features,))
successive_outputs = []
for input_t in inputs:
    output_t = np.tanh(np.dot(W, input_t) + np.dot(U, state_t) + b)
    successive_outputs.append(output_t)
    state_t = output_t
final_output_sequence = np.stack(successive_outputs, axis=0)
  
```

12

A Simple RNN

$$\text{output_t} = \text{np.tanh}(\text{np.dot}(W, \text{input_t}) + \text{np.dot}(U, \text{state_t}) + b)$$



13

A Recurrent Layer in Keras

■ Three options

- SimpleRNN, LSTM, and GRU

■ An RNN layer that can process sequences of **any** length

```
num_features = 14
inputs = keras.Input(shape=(None, num_features))
outputs = layers.SimpleRNN(16)(inputs)
```

■ Two different modes

- Return either full sequences of successive outputs for each timestep (a rank-3 tensor of shape (batch_size, timesteps, output_features))
- Return only the last output for each input sequence (a rank-2 tensor of shape (batch_size, output_features))

14

Examples for Two Different Modes

```
num_features = 14
steps = 120
inputs = keras.Input(shape=(steps, num_features))
outputs = layers.SimpleRNN(16, return_sequences=False)(inputs)
print(outputs.shape)
```

(None, 16)

```
num_features = 14
steps = 120
inputs = keras.Input(shape=(steps, num_features))
outputs = layers.SimpleRNN(16, return_sequences=True)(inputs)
print(outputs.shape)
```

(120, 16)

15

Stacking RNN Layers

```
inputs = keras.Input(shape=(steps, num_features))
x = layers.SimpleRNN(16, return_sequences=True)(inputs)
x = layers.SimpleRNN(16, return_sequences=True)(x)
outputs = layers.SimpleRNN(16)(x)
```

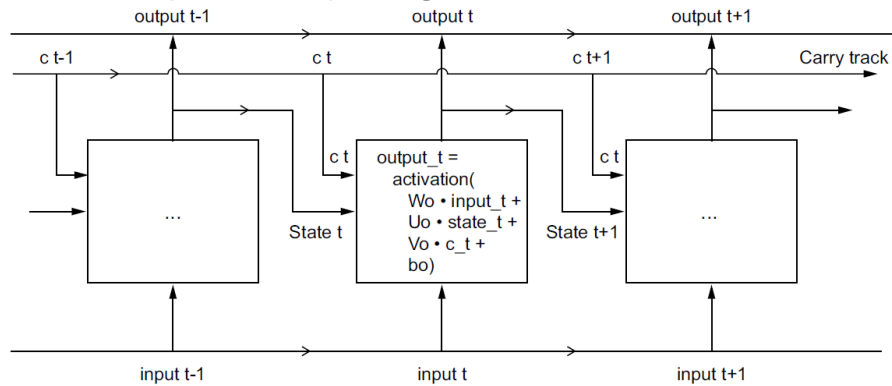
■ A major **issue** for SimpleRNN layer

- **Vanishing gradient problem**
- As keep adding layers to a network, the network eventually becomes **untrainable**
- **LSTM** and **GRU** were designed to address such an issue

16



Long Short-Term Memory (LSTM) Algorithm



$$y = activation(dot(state_t, U) + dot(input_t, W) + dot(C_t, V) + b)$$

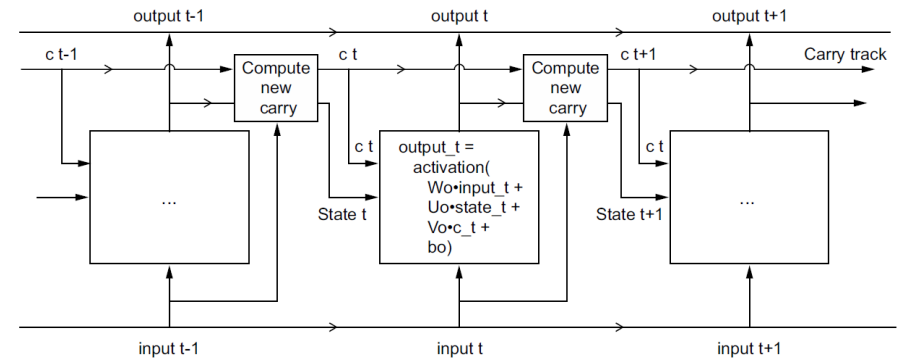
17



LSTM Algorithm

```
output_t = activation(dot(state_t, Uo) + dot(input_t, Wo) + dot(c_t, Vo) + bo)
i_t = activation(dot(state_t, Ui) + dot(input_t, Wi) + bi)
f_t = activation(dot(state_t, Uf) + dot(input_t, Wf) + bf)
k_t = activation(dot(state_t, Uk) + dot(input_t, Wk) + bk)
```

$$c_{t+1} = i_t * k_t + c_t * f_t$$



18



Outline

- Different Kinds of Timeseries Tasks
- A Temperature-forecasting Example
- Understanding Recurrent Neural Networks (RNNs)
- **Advanced Use of RNNs**

19



Advanced Use of RNNs

- Recurrent dropout
- Stacking recurrent layers
- Bidirectional recurrent layers

20

Recurrent Dropout

- This is a variant of **dropout**, used to fight **overfitting** in recurrent layers.
- The **proper** way to use dropout with a recurrent network
 - The **same** dropout mask (the same pattern of dropped units) should be applied at **every timestep**, instead of using a dropout mask that varies randomly from timestep to timestep.

21

Stacking Recurrent Layers

- This increases the **representational power** of the model (at the cost of higher computational loads).
- As long as you aren't **overfitting** too badly, you're likely under capacity.
- Increasing network capacity is typically done by **increasing** the number of units in the layers or adding more layers.
 - For instance, Google Translate algorithm was powered by a stack of **seven** large LSTM layers.

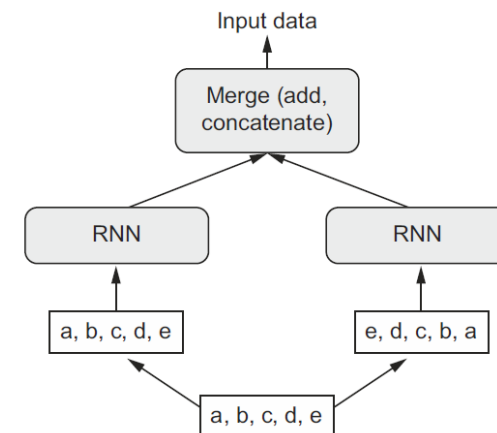
22

Bidirectional Recurrent Layers

- These present the same information to a recurrent network in **different** ways, increasing accuracy and mitigating forgetting issues.
- This is the **Swiss Army knife** of deep learning for natural language processing.
- RNNs are notably **order-dependent**: they process the timesteps of their input sequences in order, and **shuffling** or **reversing** the timesteps can completely change the representations the RNN extracts from the sequence.

23

How a Bidirectional RNN Layer Works?



24



Things to Try to Improve Performance

- Adjust the **number of units** in each recurrent layer in the stacked setup and **the amount of dropout**.
- Adjust the **learning rate** used by the RMSprop optimizer, or try a different **optimizer**.
- Try using a **stack of Dense layers** as the regressor on top of the recurrent layer.
- Improve the input to the model: try using **longer or shorter sequences** or a different **sampling rate**, or start doing **feature engineering**.

25

Were RNNs All We Needed?

Leo Feng

Mila – Université de Montréal & Borealis AI
leo.feng@mila.quebec

Frederick Tung

Borealis AI
frederick.tung@borealisai.com

Mohamed Osama Ahmed

Borealis AI
mohamed.o.ahmed@borealisai.com

Yoshua Bengio

Mila – Université de Montréal
yoshua.bengio@mila.quebec

Hossein Hajimirsadeghi

Borealis AI
hossein.hajimirsadeghi@borealisai.com

- <https://arxiv.org/abs/2410.01201>
- <https://github.com/lucidrains/minGRU-pytorch>

26



Summary

- Common-sense baseline
- Try simple models before expensive ones, to make sure the additional expense is justified
- RNNs are a great fit for timeseries data
 - LSTM layer and GRU layer
- Three methods for the advanced use of RNNs
 - Recurrent dropout
 - Stacking recurrent layers
 - Bidirectional recurrent layers

27

Q & A

Have fun!!!